



Time-Series Forecasting Pipeline Using Random Forest

Prepared by: Naimuddin
Role: Data Analytics Professional
GitHub: github.com/naimuddin | Bengaluru, Karnataka
Date: June 2026

1. Project Overview

This project implements a production-grade **Sales Forecasting Pipeline** that leverages Random Forest regression to predict daily sales volumes across multiple product-store combinations. The pipeline is built on a **Medallion Architecture** (Bronze → Silver → Gold) and delivers actionable inventory reorder recommendations, reducing overstock and stockout risk.

Business Objectives

- Forecast 30-day sales volumes per product-store with confidence intervals
- Identify seasonal and promotional demand patterns using historical time-series data
- Automate inventory reorder recommendations based on forecast + lead time logic
- Build a reusable, containerized pipeline with Docker for production deployment
- Provide interactive SQL analytics layer for business intelligence reporting

Technical Stack

Language	Python 3.11
ML Framework	scikit-learn – RandomForestRegressor (n_estimators=200, max_depth=12)
Data Processing	Pandas 2.x, NumPy 1.26
Visualization	Matplotlib 3.9, Seaborn 0.13
Database	MySQL 8.0 – Star Schema (fact + dim tables)
Containerization	Docker 24 + Docker Compose 3.9
Architecture	Medallion (Bronze / Silver / Gold)
Validation	TimeSeriesSplit 5-fold cross-validation

2. ETL Pipeline – Medallion Architecture

Bronze Layer – Raw Ingestion

Raw CSV sales data is ingested with schema validation, null detection, and clipping of negative sales values. **10,950 rows** covering 730 days × 5 products × 3 stores were generated using a realistic synthetic model incorporating trend, weekly seasonality, annual cycles, and holiday demand spikes.

Silver Layer – Cleaning & Enrichment

- Deduplication on (date, product_id, store_id) composite key
- Calendar feature derivation: year, month, quarter, week, day_of_week, is_weekend, is_month_end
- Holiday flagging: Black Friday, Christmas, New Year periods
- Revenue recalculation validation (units_sold × unit_price vs recorded revenue)

Gold Layer – Feature Engineering

- Lag features: lag_1d, lag_7d, lag_14d, lag_28d (demand history with no leakage)
- Rolling statistics: 7/14/30-day rolling mean and standard deviation
- Momentum: 7-day cumulative sales sum
- Label encoding: product_id and store_id → integer codes
- Final Gold dataset: 10,530 ML-ready rows (after lag warm-up period removed)

3. Forecasting Model

Algorithm: Random Forest Regressor

Random Forest was selected for its robustness to outliers, ability to capture non-linear seasonal patterns, and built-in feature importance ranking. Time-series integrity was maintained using **TimeSeriesSplit** (5 folds) to prevent data leakage.

Fold	MAE	RMSE	R ²	MAPE %
1	16.5	21.2	0.858	11.3%
2	25.4	39.5	0.461	24.0%
3	12.1	16.1	0.899	9.2%
4	15.3	19.9	0.883	8.7%
5	15.0	24.3	0.815	10.5%
Average	16.8	24.2	0.783	12.7%

Table 1: Cross-Validation Metrics per Fold

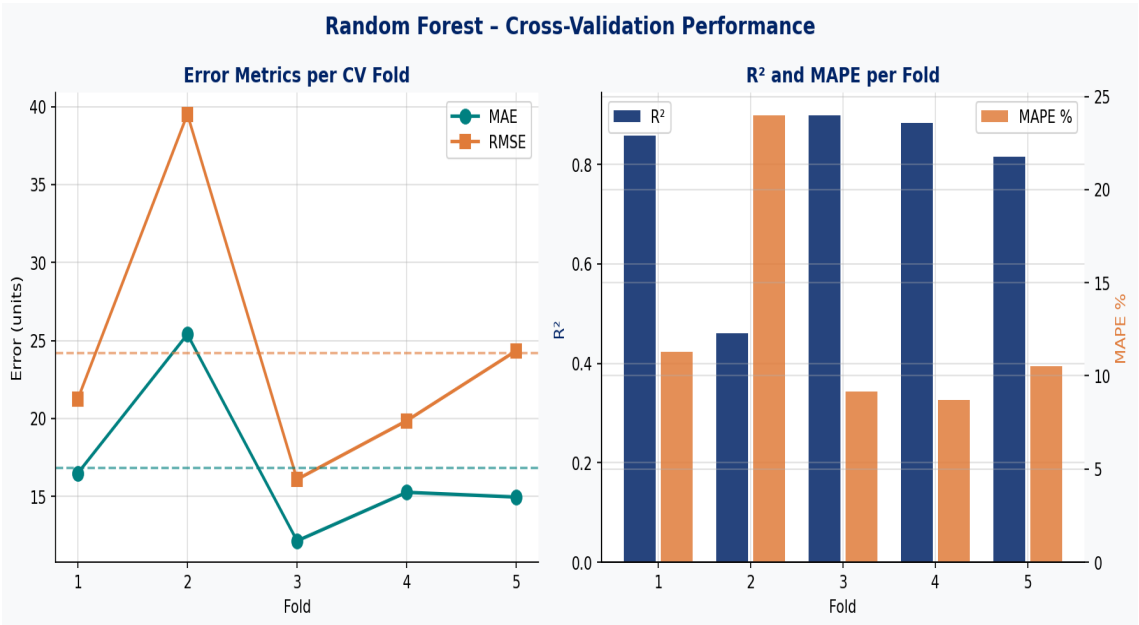


Figure 1: CV Error Metrics (MAE/RMSE) and R² per Fold

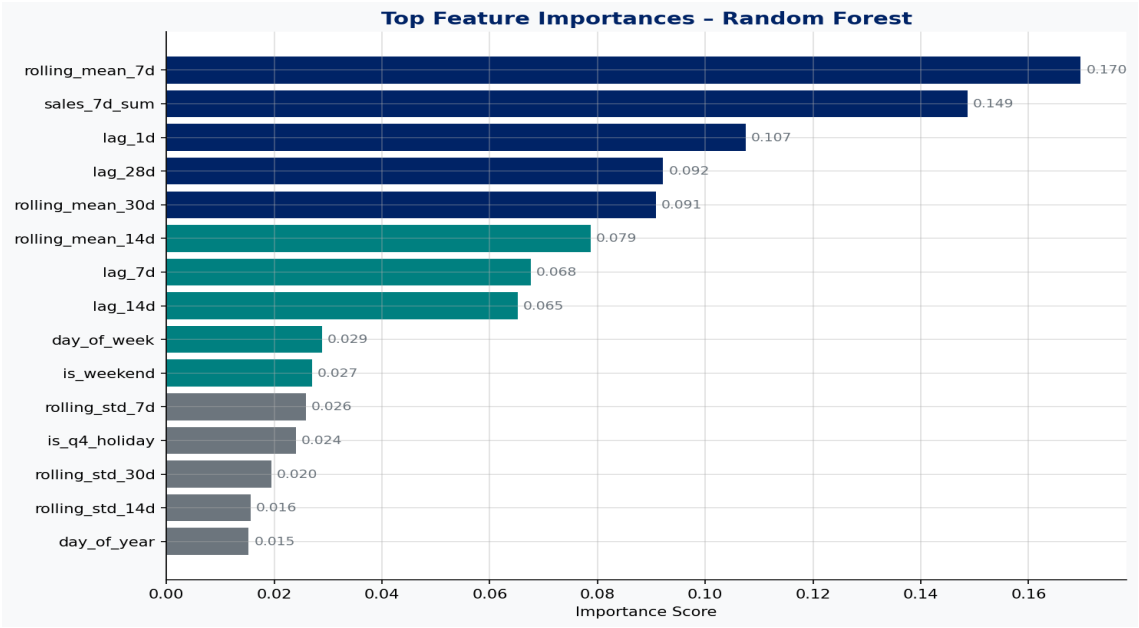


Figure 2: Top Feature Importances

4. Sales Insights & Seasonality

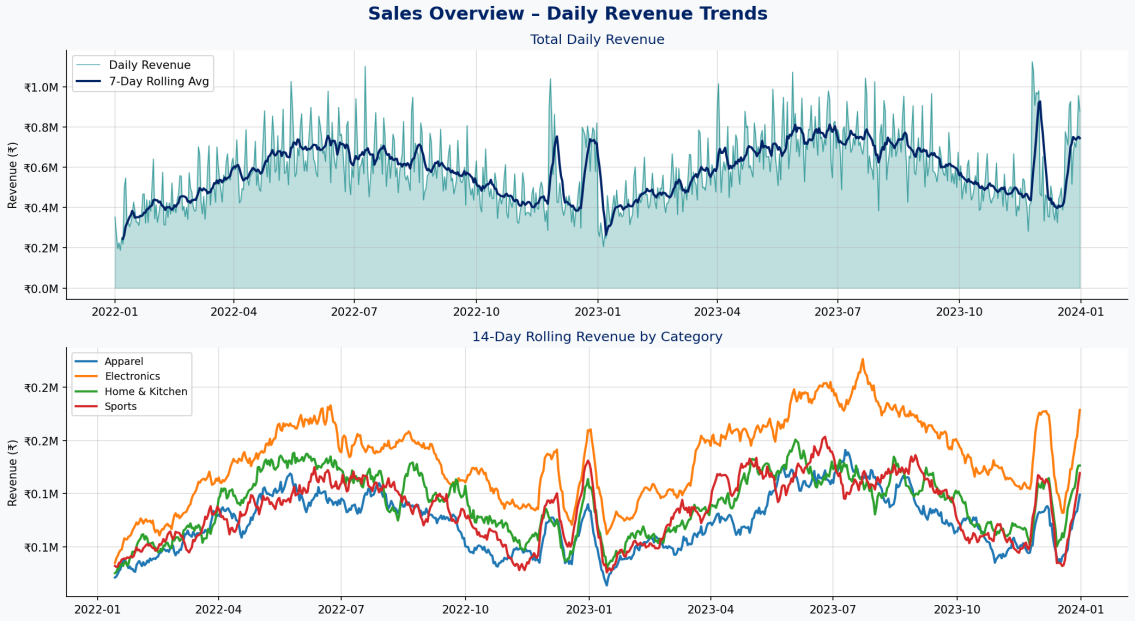


Figure 3: Daily Revenue Trend and 7-Day Rolling Average

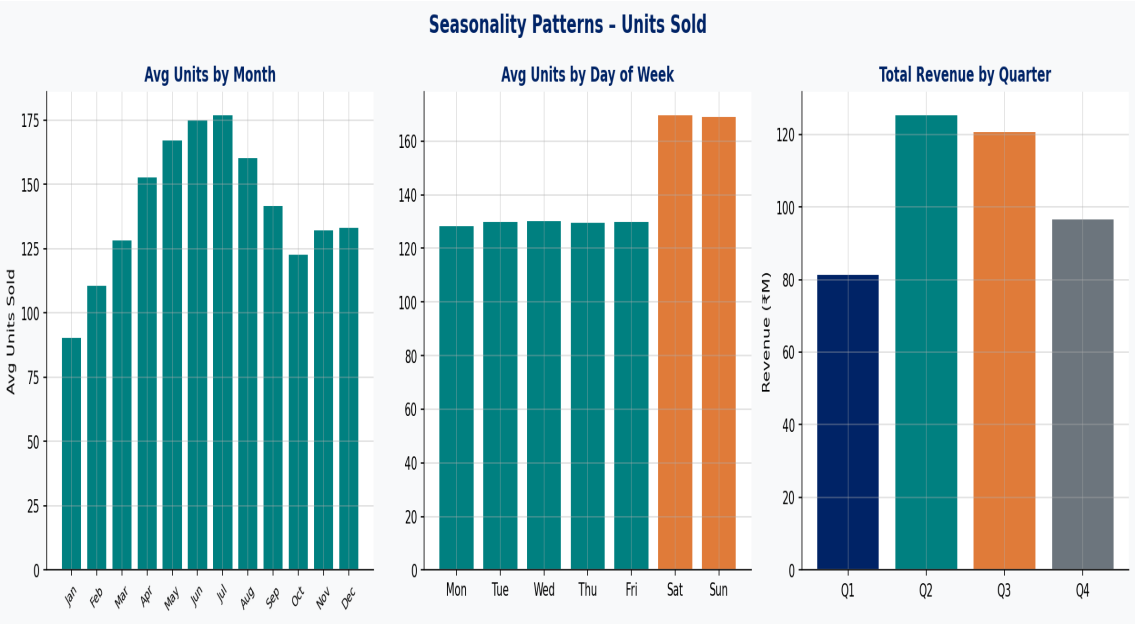


Figure 4: Seasonality by Month, Day of Week, and Quarter

5. 30-Day Forecast & Inventory Optimization

The final Random Forest model generates 30-day forward-looking forecasts with 95% confidence intervals for all 15 product-store combinations. Inventory reorder recommendations are computed by comparing the total 30-day forecast against current stock levels, accounting for safety stock buffers and supplier lead time.

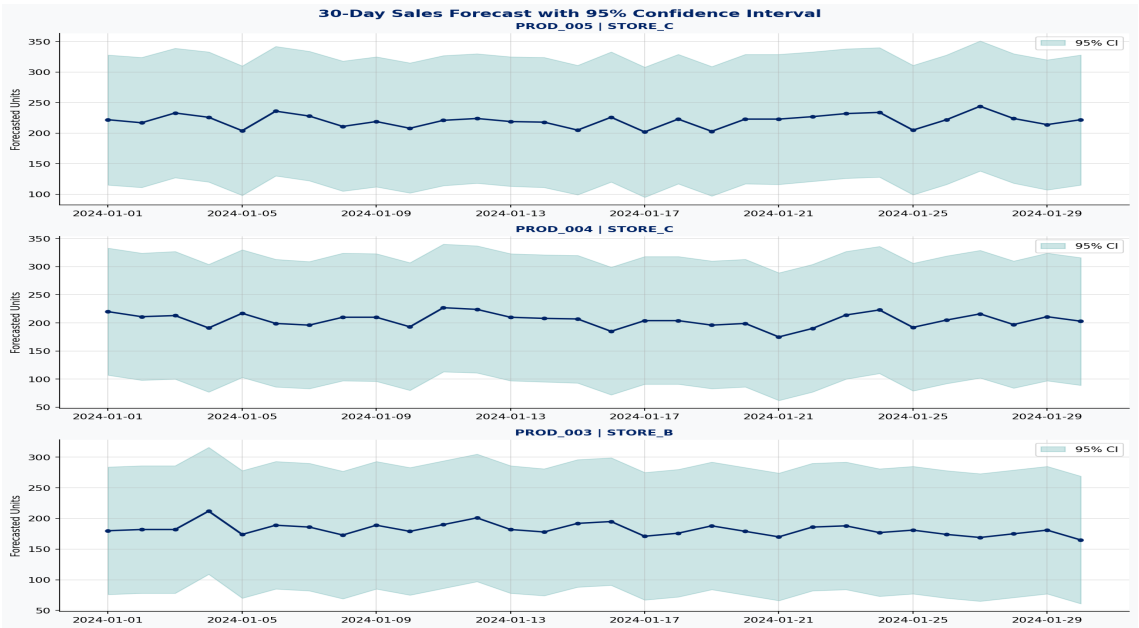


Figure 5: 30-Day Sales Forecast with 95% CI (Top 3 Product-Store Combos)

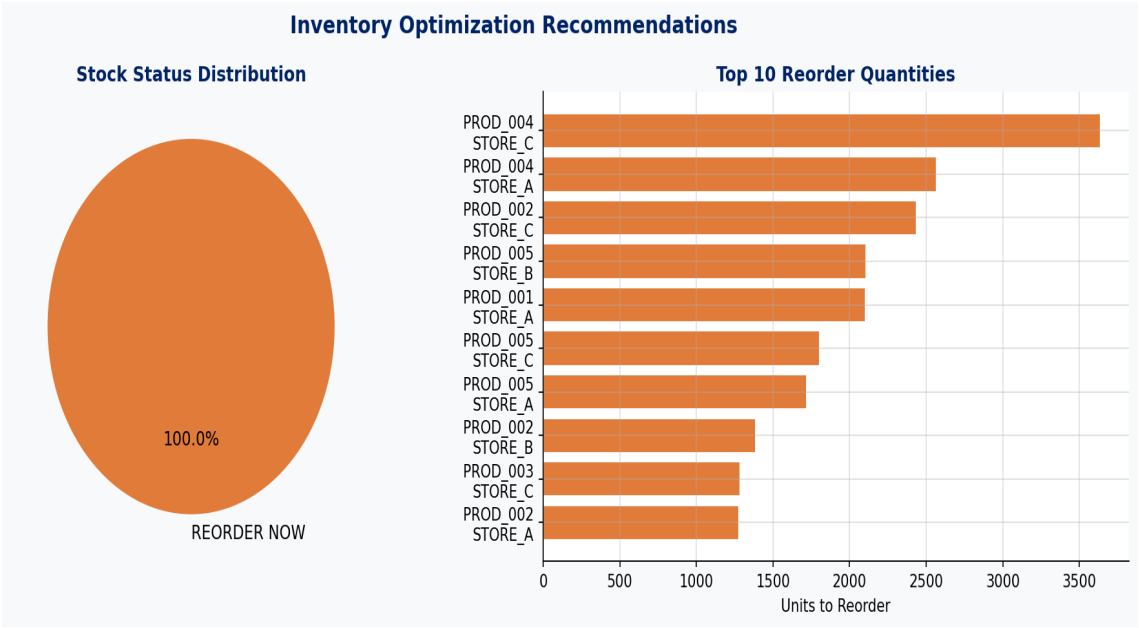


Figure 6: Inventory Reorder Recommendations

6. Database Schema & Analytics

dim_products	Product attributes – ID, name, category, unit cost
dim_stores	Store attributes – ID, region, city
dim_date	Calendar dimension with holiday and season flags
fact_sales	Daily sales transactions (units, price, revenue)
fact_inventory	Snapshot inventory levels per product-store
fact_forecasts	Model-generated 30-day forecasts with CI bounds
fact_reorder_recommendations	Inventory optimization actions

10 analytical SQL queries included:

- Monthly revenue trend by category
- Top 5 products by YTD revenue
- Week-over-week growth per store (LAG window function)
- Products below reorder point (inventory alert)
- 30-day forecast vs current stock sufficiency
- Weekend vs weekday sales comparison
- Forecast accuracy: actual vs predicted MAPE
- Quarterly store revenue ranking (RANK window function)
- Cumulative YTD revenue per product (SUM OVER)
- Holiday impact analysis (Q4 vs rest of year)

7. Deliverables & Project Structure

src/generate_data.py	Synthetic sales & inventory data generator
src/etl_pipeline.py	Bronze → Silver → Gold ETL pipeline
src/forecasting_model.py	Random Forest training, CV, forecast, inventory logic
src/visualizations.py	6 analytical chart modules
sql/schema_and_queries.sql	MySQL star schema + 10 analytical SQL queries
docker/Dockerfile	Python 3.11 slim container
docker/docker-compose.yml	Pipeline + MySQL 8.0 orchestration
outputs/	CSV exports, charts, model summary JSON
main.py	Full pipeline runner

This project demonstrates end-to-end ownership of a production ML forecasting pipeline — from synthetic data engineering and feature-rich ETL to model training with rigorous time-series CV, actionable inventory optimization, and a fully containerized deployment. The SQL analytics layer enables direct BI tool integration for stakeholder reporting.